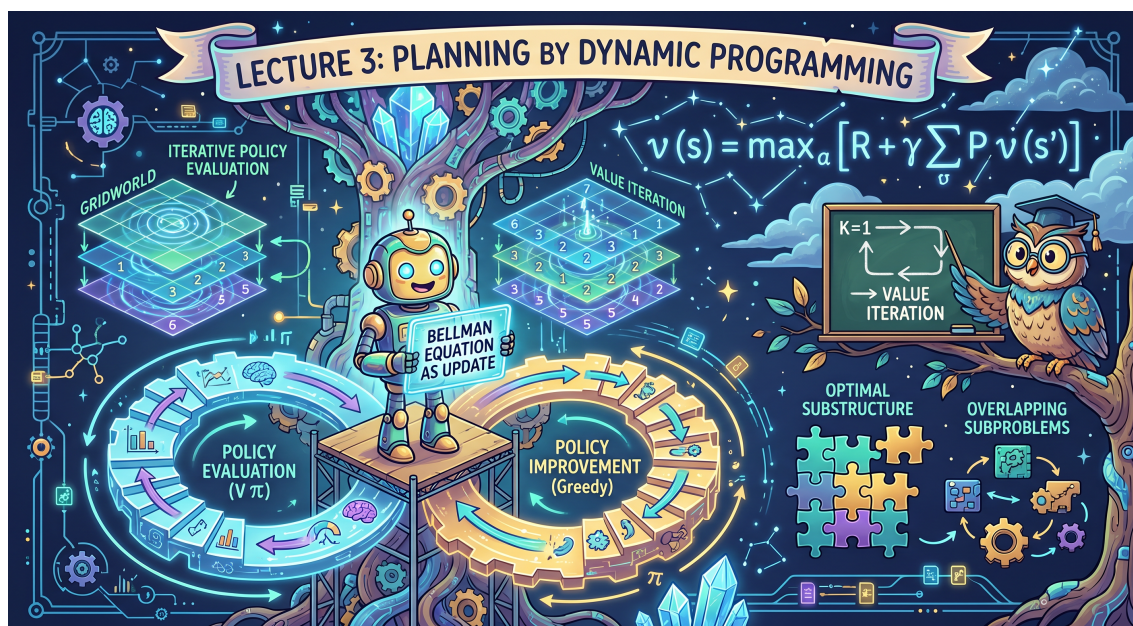


David Silver 强化学习课程笔记

Lecture3: Planning by Dynamic Programming



 SpiritsYouthHarmony

2026 年 5 月

上节课讲到的 MDP 是为 RL 中 agent 建立了相应环境，但在定义价值函数时，我们并没有说明如何求解，只是给出了一些例子（比如 Student Markov Chain），本节讲述的 policy iteration 和 value iteration 就是具体求解 MDP 的方法，其核心正是计算机科学中非常重要的算法思想——动态规划（Dynamic Programming, DP）。

1 Dynamic Programming

先来看个例子：硬币找零问题。给定一组不同面值的硬币（假设只有面值为 1、2、5 的硬币），然后给定一个目标金额。我们的目标是计算出凑到目标金额所需的最少硬币数量。用 $dp[i]$ 表示凑齐金额 i 所需的最少硬币的数量。假如现在选择了一枚面值为 m 的硬币，那此时我们要求的就是凑齐 $(i-m)$ 所需的最少硬币的数量，最后结果可以表示为 $dp[i] = \min(dp[i-m] + 1)$ ，此外，易知 $dp[0] = 0$ 。所以，可以依次计算：

- $dp[1] = \min(dp[1-1] + 1) = dp[0] + 1 = 1$
- $dp[2] = \min(dp[2-1] + 1, dp[2-2] + 1) = \min(2, 1) = 1$
- $dp[3] = \min(dp[3-1] + 1, dp[3-2] + 1) = \min(2, 2) = 2$
- $dp[4] = \min(dp[4-1] + 1, dp[4-2] + 1) = \min(3, 2) = 2$
- $dp[5] = \min(dp[5-1] + 1, dp[5-2] + 1, dp[5-5] + 1) = \min(3, 3, 1) = 1$
- ...

从这个例子，我们可以看出，动态规划其实就是一种通过把原问题分解为相对简单的子问题来求解复杂问题的方法。从 Dynamic Programming 这个短语来看，Dynamic（动态的）是指问题本身带有顺序性或时间性，是一步一步发生的；而 Programming 指的不是写代码/写程序，而是像 linear programming 里的这个 programming 一样，是一种数学意义上的“规划、求解”，那么在 MDP 的环境下，规划的最终目的就是要最优化 policy。

当然，要想使用 Dynamic Programming 求解，对问题需要有一定要求。

- 1) **最优子结构 (Optimal substructure)**：整体最优解可以被分解成子问题的最优解。例如，硬币找零问题中，要求 $dp[5]$ ，就可以转化求 $\{dp[5-1] + 1, dp[5-2] + 1, dp[5-5] + 1\}$ 中的最小值，也就是说“大问题最优解由小问题最优解决定”。
- 2) **重叠子问题 (Overlapping subproblems)**：子问题会反复出现，这样解一次缓存下来，后面重复利用才有效率。比如硬币找零问题中求出 $dp[1]$ 和 $dp[2]$ ，在计算 $dp[3]$ $dp[4]$ $dp[5]$... 时就能用到。

非常好的一点是，MDP 同时满足这两点。首先，Bellman Equation 本身就是 recursive decomposition 的方法，当前状态的价值取决于即时奖励和后继状态的价值，要求当前状态最优价值，就要求后继状态的最优价值，这是一种递归的关系，完全契合 **最优子结构** 的特点；其次，MDP 中的值函数 $v(s)$ 正是一个“缓存表”，它缓存每个状态的值，在所有父状态更新时可以直接查表复用，这正是 **重叠子问题**。

以上我们简单介绍了动态规划的基本情况，尤其是需要满足的两个特征。下面我们来看一下我们要用动态规划解决的问题——Planning（规划）。

Planning by Dynamic Programming: 假设 MDP 的所有信息**完全已知**，也就是说状态空间、动作空间、转移特性、奖励函数和折扣因子都已知，此时，我们可以用 Dynamic Programming 来解决 MDP 中的两大问题——预测和控制。注意，这只是规划问题中比较特殊的两个问题，不是所有的问题。

- 1) 预测 (prediction): 给定一个策略 π ，求它的值函数 v_π 是多少。比如说，如果 agent 要在某个网格世界里随机漫步，那么我们想知道它在这个世界里所走的每一步会得到多少奖励，以及走到哪些位置比较好。
- 2) 控制 (control): 不给定策略，直接求最优值函数 v_* 和最优策略 π_* 。也就是说，我们想知道在某个 MDP 中，agent 能做的最好的选择是什么，能获得的最大奖励是什么。

接下来的 policy evaluation、policy iteration 和 value iteration 正是在解决这两个问题。

需要说明的是，动态规划是一种广泛被使用的算法，不仅局限于 MDP 和规划问题，比如最短路径、背包问题、编辑距离、维特比算法等等都是其应用场景。

2 Policy Evaluation

如何评估一个给定的策略？即已知 MDP 和策略，我们要计算这个策略到底好不好。

- **问题的定义**：评估一个给定的策略 π ，即计算在该策略下每个状态的价值函数 $v_\pi(s)$ 。策略评估就是要计算“遵循策略 π 时，从每个状态开始能获得多少期望回报”
- **求解方法**：iterative application of Bellman expectation backup

具体来看，如式1，这是标准的 Bellman Expectation Equation。现在我们不知道 $v_\pi(s)$ ，要求这个 $v_\pi(s)$ 。我们采取的方法是如式2所示的迭代方法，在第 $k+1$ 轮迭代时，状态 s 的价值 $v_{k+1}(s)$ 是由上一轮（第 k 轮）的价值 $v_k(s)$ 代入贝尔曼期望方程求得的。非常神奇的是，随着 $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow$ ，最终的结果 $v_k(s)$ 会收敛到 $v_\pi(s)$ 。关于其收敛性的证明，可参考 [1]。

Bellman Expectation Equation

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_\pi(s') \right) \quad (1)$$

iterative application of Bellman expectation backup

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_k(s') \right) \quad (2)$$

如图1，顶部节点表示当前状态 s ，其价值函数为 $v_{k+1}(s)$ ，根据策略 π 选择动作，获得即时奖励 r ，转移到后继状态 s' ，其价值函数为 $v_k(s')$ 。需要注意的是，这里每次更新时会**同时更新所有状态的价值函数**，这叫做 **Synchronous Backups（同步备份）**。

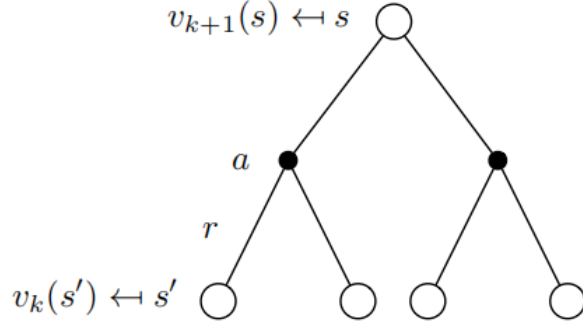


图 1: backup diagram

下面来看一个具体的例子。如图2，这是一个 4×4 的 gridworld 环境，具体细节如下：

- **状态**：共有 16 个状态，分成两种类型。图中左上角（0 号）和右下角（15 号）的阴影格子表示 terminal state；剩余的 1-14 号格子表示 nonterminal state
- **动作**：包括 $\uparrow \downarrow \leftarrow \rightarrow$ 四个可选动作
- **环境建模**：Undiscounted MDP ($\gamma = 1$)
- **奖励**：除了 terminal state，剩下的每走一步给一个 **reward=-1**
- **策略**：随机策略，即 $\pi(\uparrow|\cdot) = \pi(\downarrow|\cdot) = \pi(\leftarrow|\cdot) = \pi(\rightarrow|\cdot) = 0.25$

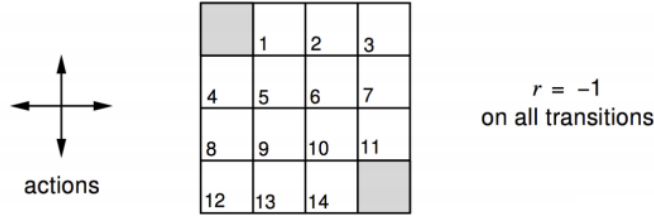


图 2: a small gridworld

下面我们来迭代计算当前策略下的 $v_\pi(s)$ 。

- **k=0**：所有状态的值初始化为 0
- **k=1**：以红色方框标出的这个格子为例（其序号为 1），首先，策略是随机的，意味着当前状态下均匀概率朝上下左右四个方向走，无论往哪走，因为这个格子不是 terminal state，所以它都会获得一个 **reward=-1**，而它的后继状态分别是 0 号、1 号（自己）、2 号和 5 号，这几个状态在 $k=0$ 的 $v_0(s)$ 都是 0。所以，计算式为 $v_1(\text{grid}_1) = 0.25 \times (-1 + v_0(\text{grid}_0)) + 0.25 \times (-1 + v_0(\text{grid}_1)) + 0.25 \times (-1 + v_0(\text{grid}_2)) + 0.25 \times (-1 + v_0(\text{grid}_5)) = -1$
- **k=2**：仍以红色方框标出的这个格子为例，这一轮后继状态的价值要用 $k=1$ 的 $v_1(s)$ ，所以，计算式为 $v_2(\text{grid}_1) = 0.25 \times (-1 + v_1(\text{grid}_0)) + 0.25 \times (-1 + v_1(\text{grid}_1)) + 0.25 \times (-1 + v_1(\text{grid}_2)) + 0.25 \times (-1 + v_1(\text{grid}_5)) = 0.25 \times (-1 + 0) + 0.25 \times (-1 - 1) + 0.25 \times (-1 - 1) + 0.25 \times (-1 - 1) = -1.75$

•

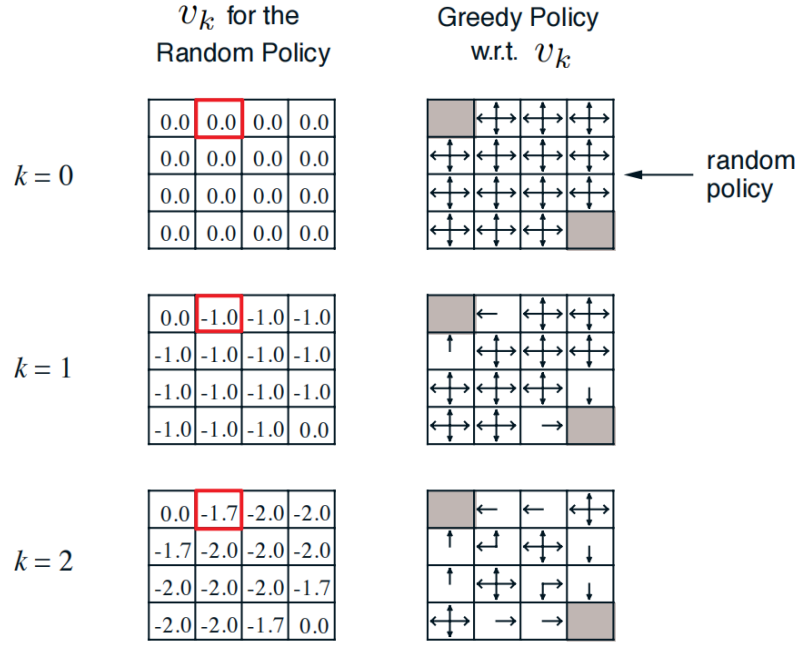


图 3: Iterative Policy Evaluation in Small Gridworld

• $k=\infty$: 如图4, 随着迭代不断进行下去, 最后每个状态的值都会等于 $v_\pi(s)$

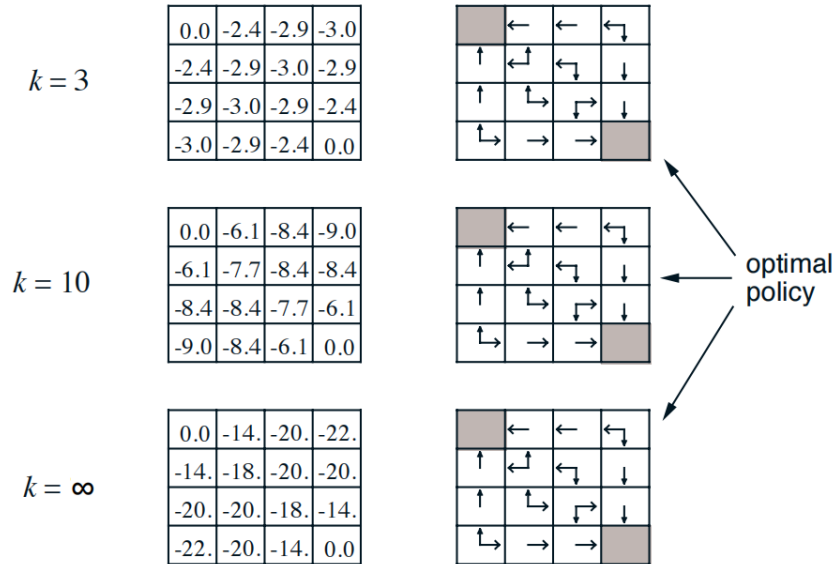


图 4: Iterative Policy Evaluation in Small Gridworld

从最后计算得到的结果来看, 离 terminal state 越远的那些格子价值越低, 这完全符合我们的直觉。

此外，在图3和图4中，每轮迭代的过程中，我们都给出了相应的 **Greedy Policy**，也就是说每轮迭代完以后，我们对于 $\uparrow \downarrow \leftarrow \rightarrow$ 这四个动作的好坏都会有新的判断。比如 $k = 1$ 迭代计算完后，红色框对应的状态就知道：往左边走能到 $v_1(\text{grid}_0) = 0$ 的这格，其他方向走都只能到 $v_1(\dots) = -1$ 的这些格子，所以肯定要往左边走。这其实就是接下来要讲的 Policy Iteration 的核心。

3 Policy Iteration

Policy Evaluation 告诉我们在某个策略下，每个状态的价值是多少，接下来，我们要解决的问题是：如何改进策略？刚刚的例子其实已经告诉我们答案了：贪心策略。

基本的过程是这样的：

- 给定策略 π
- **Evaluation**: $v_\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma R_{t+2} + \dots | S_t = s]$
- **Improve the policy by acting greedily with respect to v_π** : $\pi' = \text{greedy}(v_\pi)$

这种思想其实很简单，每一轮计算出 $v_\pi(s)$ 以后，我们其实就知道不同的动作能带来的价值了，那么直接选那些能带来最大价值的动作就行了。

在 gridworld 的例子中，迭代到最后，也就是 $k = \infty$ 时，按照上面所说的贪心策略做一次 improve，实际上我们就得到了 optimal policy，即 $\pi' = \pi^*$ 。

但在更复杂的问题里，通常需要进行多次 “Evaluation -> Improvement -> Evaluation -> Improvement ...” 这样的循环迭代，而且最终一定会收敛至 optimal policy π^* ，这个过程就是所谓的 Policy Iteration。

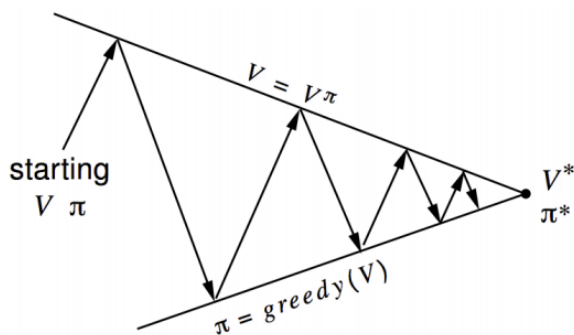


图 5: Policy Iteration

如图5所示，Policy Iteration 是一个不断迭代的过程：

- **起始点**：给定初始策略 π 和初始价值 V 。
- **Evaluation (向上箭头)**：固定策略 π 不变，通过迭代计算，让价值 V 逼近真实的 V_π (即 $V = V_\pi$)。

- **Improvement (向下箭头)**: 基于当前的价值 V , 采用贪心策略更新 π 。显然, 新策略 π' 的价值一定不低于旧策略 ($\pi' \geq \pi$), 所以在策略空间上向右 (向最优方向) 移动了一步。
- **迭代**: 这个“上-下-上-下”的锯齿过程不断重复, 最终收敛到 (v^*, π^*) , 即最优价值函数和最优策略。

下面来看一个经典的例子——Jack's Car Rental。Jack 经营着两家汽车租赁店, 他需要决定每天晚上如何在两家店之间调配车辆, 以最大化利润。

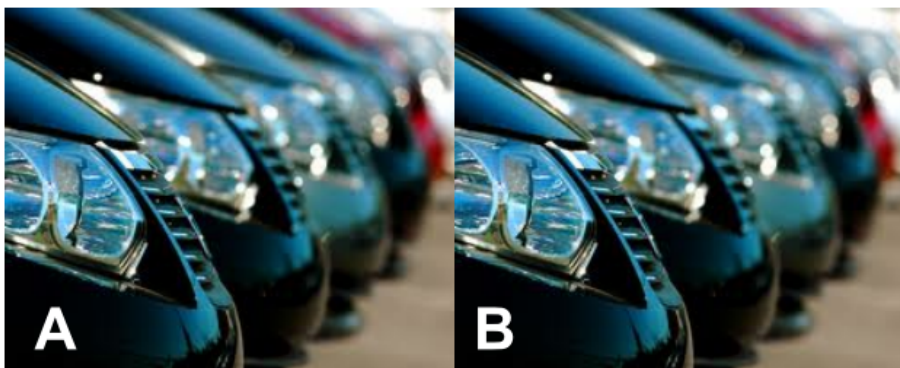


图 6: Jack's Car Rental

- **States**: 如图6, 有两个地点 A 和 B, 每个地点最多能停 20 辆车。状态可以用 (n_A, n_B) 表示, 其中 n_A 是地点 A 的车辆数, n_B 是地点 B 的车辆数。总共有 $21 \times 21 = 441$ 个可能的状态。
- **Actions**: 每天晚上 Jack 可以在两个地点之间移动车辆, **最多移动 5 辆**。也就是说, 动作 a 的取值范围是 $[-5, 5]$ 。 $a > 0$ 表示从地点 A 移到地点 B a 辆车; $a < 0$ 表示从地点 B 移到地点 A $|a|$ 辆车; $a = 0$ 表示不移动。
- **Reward**: 每租出一辆车赚取 10 美金。
- **Transitions**: 租车和还车的数量 n 服从泊松分布。 $P(n) = \frac{\lambda^n}{n!} e^{-\lambda}$, 其中 λ 是平均值。
 - A: 平均每天租出 3 辆, 平均每天还回 3 辆。
 - B: 平均每天租出 4 辆, 平均每天还回 2 辆。

使用 Policy Iteration 求解这个问题的过程如图7所示。可以看到, 经过 4 轮迭代, 已经获得了 optimal policy (第三轮和第四轮之间的差距非常小, 可以认为收敛)。

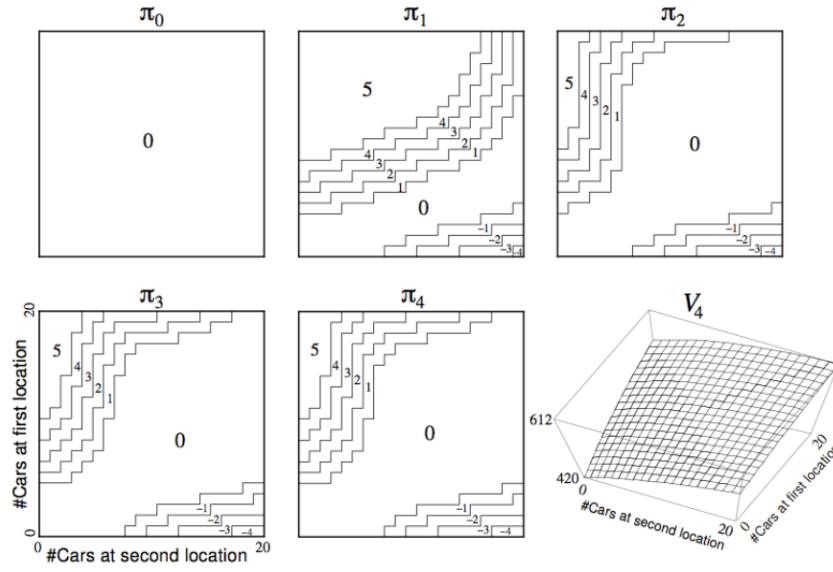


图 7: Policy Iteration in Jack's Car Rental

下面来解答一个问题：虽然从直觉上来说，贪心确实能让策略越来越好，但如何从数学上保证这么做就一定能收敛到 optimal policy？

Policy Improvement

只要我们在每个状态下都选择当前看来最好的动作（贪心策略），那么新策略一定比旧策略好（或者一样好）；当无法再改进时，我们就找到了最优策略。

我们以确定性策略 $a = \pi(s)$ 为例，即在状态 s 下，固定选择某个动作 a 。那么，贪心策略就可以表示为

$$\pi'(s) = \arg \max_{a \in A} q_{\pi}(s, a) \quad (3)$$

也就是说，在当前策略 π 下，找到状态 s 可选动作 a 中能让 $q_{\pi}(s, a)$ 最大的那个，固定选择这个动作，以此作为新的策略。

显然， $q_{\pi}(s, \pi'(s)) = \max_{a \in A} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s))$ 。而根据值函数的定义，当前策略下状态 s 的价值就等于按照策略 π 选择动作能获得的累计奖励，由于是确定性策略，所以在状态 s 采取的动作就是 $\pi(s)$ ，那么 $q_{\pi}(s, \pi(s)) = v_{\pi}(s)$ ，可得

$$q_{\pi}(s, \pi'(s)) = \max_{a \in A} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s) \quad (4)$$

$$\begin{aligned} v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) = \mathbb{E}_{\pi'} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, \pi'(S_{t+2})) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s] = v_{\pi'}(s) \end{aligned} \quad (5)$$

一些补充说明：

- $q_\pi(s, \pi'(s)) = \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$ $q_\pi(s, \pi'(s))$ 表示在状态 s 下，按照新策略 π' 先走一步，获得即时奖励 R_{t+1} ，然后到达下一个状态 S_{t+1} 。从下一状态开始，还按旧策略 π 来估计价值。
- $E_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s]$
根据式4，可得 $v_\pi(S_{t+1}) \leq q_\pi(S_{t+1}, \pi'(S_{t+1}))$
- 下面的步骤其实就是重复上面两步，按照 $q_\pi(s, \pi'(s))$ 的定义写开，然后应用式4的结论即可。

$v_\pi(s) \leq v_{\pi'}(s)$ ，这说明，贪心策略不仅会让当前这一步选择的动作的 q 值增加，从长期来看，策略 π' 下状态 s 对应的价值函数 $v_{\pi'}(s)$ 也会增加。

随着贪心策略不断迭代进行下去，最终会出现 $q_\pi(s, \pi'(s)) = \max_{a \in A} q_\pi(s, a) = q_\pi(s, \pi(s)) = v_\pi(s)$ 这种情况。这也就意味着新策略 π' 选出来的动作，和旧策略 π 原本选的动作，在价值上已经一样了，improvement 停止。而 $v_\pi(s) = \max_{a \in A} q_\pi(s, a)$ 正是 Bellman optimality equation，这也就说明 $v_\pi(s) = v_*(s)$ ，策略 π 是 optimal policy。

回看一下图3和图4中的例子，我们发现，其实 $k = 3$ 时 policy improvement 得到的已经是 optimal policy 了。所以，我们在做 policy iteration 的时候其实没必要非得等到 policy evaluation 收敛再进行 policy improvement，这引出了所谓的 Modified Policy Iteration。

Modified Policy Iteration

1. ϵ -convergence of value function
2. k-iteration

方法 1——“ ϵ -convergence of value function”，简单来说，就是设定一个阈值 ϵ ，只要相邻两次迭代之间价值函数的变化量小于这个数，我们就认为“差不多够了”，停止评估，直接进入下一步的策略改进。

方法 2——“k-iteration”，更加简单粗暴，设定一个 k ，评估时只迭代 k 轮，不管有没有收敛都进行策略改进。（如果 $k=1$ ，这种方法就是接下来要提到的 Value Iteration……

4 Value Iteration

回顾一开始提到的动态规划的核心，即最优子结构和重叠子问题，在求解最优策略时，我们可以用最优性原理（Principle of Optimality）来描述“最优子结构”：

Theorem (Principle of Optimality)

A policy $\pi(a|s)$ achieves the optimal value from state s , $v_\pi(s) = v_*(s)$, if and only if

- For any state s reachable from s'
- π achieves the optimal value from state s' , $v_\pi(s') = v_*(s')$

也就是说，任何最优策略（Optimal Policy）都可以被拆解为两部分：

- 第一步的最优动作 A_* ：在当前状态下，必须做出最好的动作选择。

- 后续的最优策略：在执行完第一步到达新状态 S' 后，从 S' 开始往后的所有策略也必须是最优的。

我们基于这个 Principle of Optimality 来构建 value iteration 的算法：

- **核心：利用子问题的解**

- 如果我们已经知道了所有“后继状态” s' 的最优价值 $v_*(s')$ （即解决了子问题），那么，当前状态 s 的最优价值 $v_*(s)$ 就可以通过 one-step lookahead（一步前瞻）算出来，如式6

$$v_*(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \right) \quad (6)$$

和 policy iteration 一样，value iteration 也是采用迭代的方式更新 v_k 直至逼近 v_*

$$v_{k+1}(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right) \quad (7)$$

Value iteration（价值迭代）其实就是不断地重复应用7这个公式来更新价值函数。而且从直觉上来说，这个式子的更新过程是“work backwards”的，我们先知道终点（或接近终点）的价值，然后倒着推，算出前一步的价值，再算前前一步……直到推算出起点的价值……

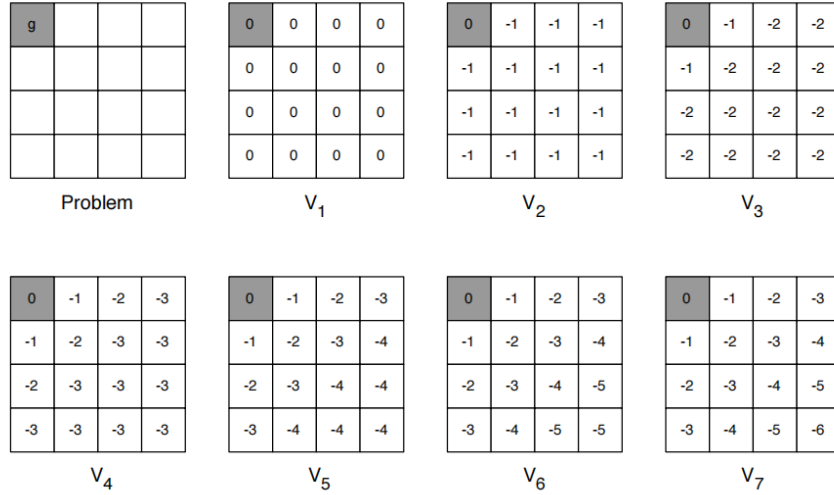


图 8: Example: Shortest Path

如图8，这是个求最短路径的例子，我们来看看怎么计算的：

- 左上角阴影格子是目标位置，没有到达目标位置时每走一步得到-1 的奖励
- 初始时所有状态的值均设为 0
- 第一次迭代，对于目标格子自己来说，往左和往上走保持不动，不拿任何奖励，而往下或往右会得到-1 的奖励，根据 value iteration 取最大的计算方法，其 value 更新后为 0；再拿第一行第二列的格子为例，无论往哪走都会得到-1 的奖励，而且后续状态的价值都是 0，所以其 value 更新后为-1

- 第二次迭代，拿第一行第二列的格子来说，会发现往左走得到-1 的奖励，后继状态的 value 是 0，比其他三个的-1 都大，所以 value 更新后还是-1；但是对第二行第二列的格子，无论往哪走都会得到-1 的奖励，而且后续状态的价值都是-1，所以其 value 更新后为-2
- ...

这样更新下去，最终能得到所有格子的 value，也就知道了所有格子怎么走才能最快抵达目标位置。而且观测更新过程可以发现，value 的更新确实是 “work backwards”，像波浪一样从左上角的目标位置向右下角的最远点不断更新。

下面给出 Value Iteration 的完整概念：

- 目标：直接找最优策略 π
- 求解方法：iterative application of Bellman optimality backup
- $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_*$

Value Iteration 与 Policy Iteration 的区别：

- 没有显式的策略：在策略迭代中，我们明确地维护一个策略 π ，先评估它，再改进它。但在价值迭代中，我们只更新价值函数 v ，中间过程并不维护一个具体的策略。
- 中间价值可能无意义：在收敛之前，中间算出来的 v_k 可能并不代表任何具体策略的价值，它只是一个向最优值逼近的数值。只有当算法收敛到 v_* 后，我们才能从中提取出最优策略。

事实上，Value Iteration 就是 $k = 1$ 时的 Policy Iteration。简单来说，Policy Iteration 包括两部分，先是进行 policy evaluation，迭代 k 轮直至收敛，以求得真实的 $v_\pi(s)$ ，然后再根据计算得到的 $v_\pi(s)$ 采取贪心策略以更新策略。那么，如果 $k=1$ ，policy Iteration 其实就是用 bellman expectation equation 算一次 value，然后直接取最大的（贪心策略），这正是 value iteration 干的事，直接把最大的 immediate reward + successor state value 拿过来当成自己的 value。

总结一下：标准策略迭代是“评估到收敛”才去改进策略，而价值迭代相当于每做一次备份（ $k=1$ ）就立刻改进一次。因为价值迭代公式里的 $\max$ 操作，本质上就是把“选最好动作（策略改进）”和“算一步价值（策略评估）”合并在了一起，不再等待收敛。

参考文献

- [1] David Silver. *Lectures on Reinforcement Learning*. URL: <https://davidstarsilver.wordpress.com/teaching/>, 2015.
- [2] 动态规划硬币找零问题可视化. URL: <https://gallery.selfboot.cn/zh/algorithms/dpcoin>, 2025.